

National College of Ireland

Project Submission Sheet

Student Name: VARUN GAMPA
Student ID: 23398639
Programme: MSc In Cloud Computing **Year:** 2026 - 2027
Module: Research in Computing
Lecturer: Sean Heeney
Submission Due Date: 7th August, 2027
Project Title: Predictive Storage Cost Optimization Framework for Amazon S3: Intelligent Storage-Class Recommendation, Forecasting and Savings Estimation
Word Count: 3864

Academic Integrity Certificate Verification Code:

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.

ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting Monday, 19th January 2026, all students are required to successfully complete the Epigeum Academic Integrity Course, which is available on Moodle. This is a self-paced online course consisting of 4 modules and will take approximately 2 hours to complete. Once you have successfully completed the course, you will receive a Certificate & Verification Code. This code must be included on the Project Cover Sheet when submitting your work. For full details and guidance, please visit: <https://libguides.ncirl.ie/academicintegrity/epigeumforstudents>

Signature: VARUN GAMPA
Date: 7th AUGUST 2027.

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.

3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplements in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

RESEARCH IN COMPUTING

Predictive Storage Cost Optimization Framework for Amazon S3: Intelligent Storage-Class Recommendation, Forecasting and Savings Estimation

Your Name/Student Number	Course	Date

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click [here](#).

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

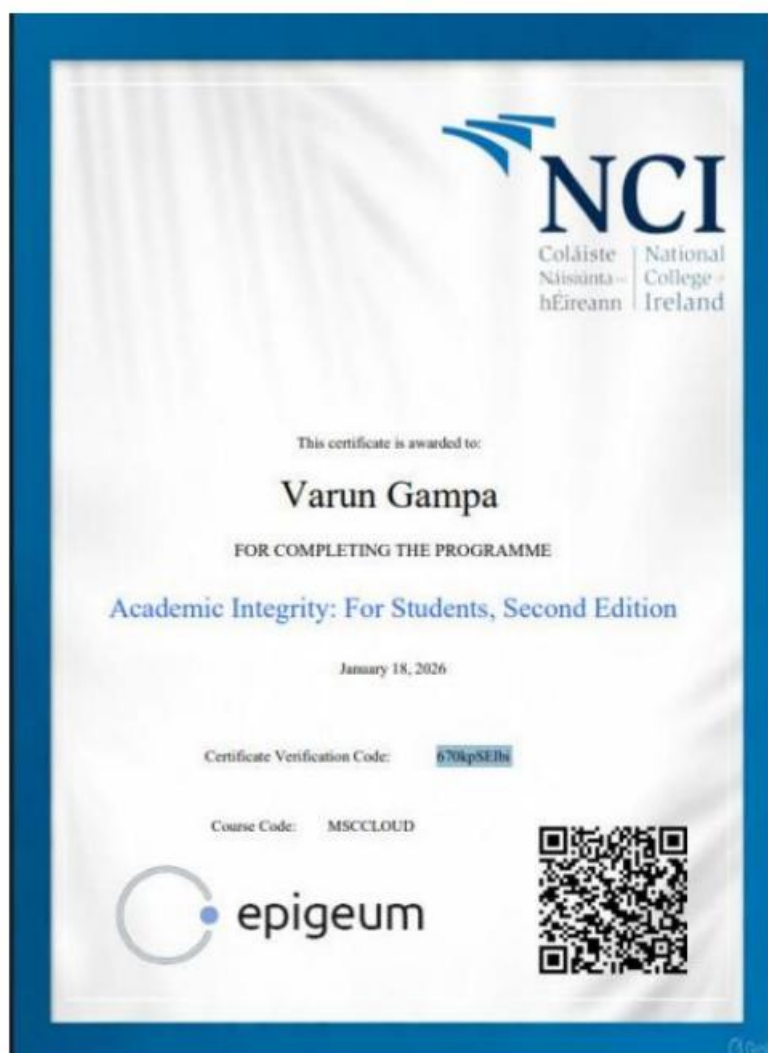
This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used.**

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Academic Integrity Acknowledgement Supplement



Predictive Storage Cost Optimization Framework for Amazon S3: Intelligent Storage-Class Recommendation, Forecasting and Savings Estimation

Research in Computing – CA2

Varun Gampa
X23398639@student.ncirl.ie
MSc in Cloud Computing
National College of Ireland

Abstract

Operating cost of the cloud is now a significant factor and Amazon S3 has reactive features: Lifecycle Policies are based on age, and Intelligent-Tiering is only triggered when inactivity is detected, but neither feature makes an assumption about what it will cost you to keep the data on the cloud, nor does it estimate savings before the feature kicks in. We propose a Predictive Storage Cost Optimization Framework that combines the three components of the storage-class recommendation, time-series cost forecasting, and savings estimation into a single AWS-native pipeline, run across real S3 data instead of simulated data. It is novel in combining three functions that AWS currently provides as separate and unreconciled services (Lifecycle Policies, Intelligent-Tiering and Cost Explorer) under one roof and auditable tool to make decisions, while also keeping a naive forecasting baseline to prevent claims of accuracy that are too good to be true. The framework will be tested with accuracy of storage class allocation, forecast error, and the savings achieved using it as compared to AWS-native storage tiers, to give empirical evidence of the value of proactive vs reactive management of S3 storage for cloud FinOps practice.

Keywords: Cloud Storage Optimization, Amazon S3, Predictive Cost Forecasting, Intelligent Tiering, FinOps, AWS, combined framework.

1 Introduction

As this adoption continues to grow, storage cost is one of the biggest, recurring components of cloud operating budgets, as organisations increasingly turn to Amazon S3 to store their growing data volumes, particularly those that are large (Liu et al., 2025). The S3 family of storage classes range from Standard (fastest retrieval time) to Glacier Deep Archive (slowest retrieval time), allowing for a balance of storage costs and retrieval speed. The number of objects and the access patterns change, and it's hard to determine if the proper class is selected for each object manually. Both of AWS's native solutions (Lifecycle Policies and Intelligent-Tiering) are reactive: Lifecycle Policies move objects to a colder tier after a certain point, even if that point is not based on actual future access¹, and Intelligent-Tiering moves an object to a colder tier after a period of inactivity has been detected². Both are non-predictive: neither predicts what an object's storage cost will be next month, nor estimates the amount of objects that will be saved if a given optimisation action is taken. This reactive vs proactive cost management gap is the area of focus for this research.

Storage cost management is important, as FinOps - the practice of balancing cloud costs and busi-

¹<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html>

²<https://aws.amazon.com/s3/storage-classes/intelligent-tiering/>

ness value is now an organisational priority (Bodra, 2025; Yakkanti, 2025), and machine learning is increasingly suggested as one of the means of making better resource allocation decisions. From the infrastructure side, the recent systems research work confirms the above: Yang et al. (2025) demonstrate that the observed improvements in storage efficiency in a warehouse-scale system can be measured with ML-driven placement decisions; and Shen et al. (2025) and Tu et al. (2025) demonstrate that dynamic placement of tiers based on workload outperforms static placement. Each, however, fails to show the storage-class recommendation, cost estimating and savings calculation integrated in a deployable pipeline for one particular, commonly used service like Amazon S3 – capabilities that found further expression in the research niche explored in Section 2.

Research Question

To what extent does an integrated predictive storage-class optimisation framework – combining machine-learning-based storage-class recommendation, time-series-based cost forecasting, and savings estimation – reduce Amazon S3 storage cost and improve storage-class allocation accuracy, relative to AWS-native Lifecycle Policies and Intelligent-Tiering, when evaluated in a live AWS environment?

Proposed Solution

The research suggests a five-step framework, implemented entirely on AWS: (1) a metadata collection module on top of the AWS SDK (Boto3) that collects attributes of individual objects; (2) a hybrid rule-based and machine-learning based recommendation engine that proposes the most cost-efficient storage class for every object; (3) a time-series forecasting module that estimates the future storage cost; (4) a savings-estimation component that quantifies the expected savings of each recommendation before applying it; and (5) an evaluation harness that compares the decisions of the proposed framework with those of Lifecycle Policies and Intelligent-Tiering on the same bucket contents.

Contribution

Today, Lifecycle Policies, Intelligent-Tiering and Cost Explorer are three separate AWS services, and an administrator has to reconcile them manually referencing tiering decisions to a completely different console for cost reporting that does not have a shared data model between the two. This research brings together storage-class recommendation, forward cost forecasting and savings estimation into a single decision-support pipeline, rather than having three signals, one for each, to interpret by hand, and validates the research end-to-end on the live infrastructure of S3 instead of through simulation.

More specifically, there are three contributions. First and foremost, a technical contribution; a re-usable, meta-annotation-based recommendation engine that, in addition to the deployment-strategy evaluation that is natively supported, also enables a supervised classifier that can be deployed alongside it, and thus provide a more active recommendation capability than a mere comparative benchmark. Second, a methodological contribution: a protocol to evaluate forecast accuracy as compared to the naive persistence benchmark that has seldom been considered in applied cloud-cost studies and which is identified by Beck, Doern and Vogl (2025) as a risk of inflated forecast accuracy. Third, a practical contribution: empirical, quantified evidence simulated projections as well as realised cost savings, allocation accuracy – that shows the benefits of proactive, forecast-driven storage-tier management over the reactive native tooling AWS provides, with clear relevance to FinOps practitioners who are evaluating whether spending time on such tooling is worthwhile.

This work is the first to combine and test all three functions jointly for Amazon S3 empirically, while each of the previous works optimise placement, cost and/or predictive FinOps in isolation.

Document Structure

Section 2 critically reviews the literature on AWS-native and research-grade storage optimisation, ML-driven storage tiering, cost-optimised storage architectures, and cloud cost forecasting, before identifying the specific research niche this work occupies. Section 3 details the proposed research method: system architecture, dataset and tooling, project plan with Gantt chart, evaluation plan, and ethical considerations.

2 Literature Review

2.1 Reactive AWS-Native Storage Optimisation

AWS has two key reactive storage optimisation features: at the extremes of the spectrum. Lifecycle Policies are calendar-based rules applied to any object, are deterministic and do not depend on the use. With Intelligent-Tiering, you don't need to manually designate access tiers for objects, because it monitors access activity and automatically moves objects between tiers based on the time they haven't been accessed. The drawbacks of both Lifecycle Policies and Intelligent-Tiering are an inability to know what is actually being used, and an inability to act proactively once a pattern has shifted. Neither is addressed by the other; the ability to reason about Lifecycle Policies is lost by not knowing usage, and the ability to respond in real-time isn't possible with Intelligent-Tiering because it only responds in retrospect once a pattern has changed. Both mechanisms are therefore retrospective: they don't predict how an object will be accessed in the future or the number of objects that will be saved by a transition. This is exactly what the research question in Section 1 requires to assess if an integrated framework can add, and under this argument lies the motivation for the development of the case for predictive and forward-looking decision support throughout this review. AWS' current tools are limited to simply following the rules, so the next question is whether there's an existing systems research approach to solving the forward-looking, learned storage placement problem, and if there is, whether it can be adapted to an Amazon S3 context. This is dealt with specifically in section 2.2.

2.2 Machine-Learning-Driven Storage Placement and Tiering

"Bring your own model" storage placement in WsC: cross-layer optimization for efficiency metrics is an application-level approach presented by Yang et al. at MLSys 2025, which is strong on infrastructure-scale placement, but is not optimized for consumer-facing storage services like Amazon S3 and is not predictive of cost. Tu et al. (2025) also demonstrate the benefits of optimising the storage-hierarchy throughput and tail latency with a mirror-optimised tiering strategy (MOST) in a storage-hierarchy system, with the optimization of load rebalancing across tiers, but their evaluation is limited to an internal caching-hierarchy instead of a public cloud storage-class model with true per-class pricing. Optimised directly, Yang et al. make a placement decision based on optimisation of which application-level model to use, whereas Tu et al., make a placement decision based on optimisation of load-balancing once a tiering decision is made; both of these papers report a quality metric and neither explicitly report a dollar figure on which a practitioner can act. Together, both works show that a learned placement is superior to static placement rules or reactive placement rules – directly relevant to the research question's premise that an actionable and costed recommendation can be provided by pointing at a live Amazon S3 bucket – but both do not recommend a placement algorithm that a cloud administrator could use for recommending an actionable and costed placement in a real Amazon S3 bucket. This is an immediate focus of this proposal. Hence the learned placement is identified as being superior to static rules at the infrastructure level but its translation to a costed per-object recommendation for a specific public-cloud service is left open. A related but separate line of inquiry is: If a placement or tiering decision can be made, how to optimize it for "cost" specifically, and has this been done for storage services like Amazon S3? The next section (2.3) focuses on this literature.

2.3 Cost-Optimised and Workload-Aware Storage Architectures

Shen et al. (2025) propose TierBase, an IEEE International Conference on Data Engineering (ICDE 2025) (CORE: A*) published workload-driven, cost-optimised key-value store developed by Ant Group which dynamically synchronises data between storage classes and cache. Liu et al. (2025) tackle a complementary problem with SkyStore, published at Proceedings of the VLDB Endowment (VLDB 2025) (CORE: A* published independently from any single professional body, not IEEE), optimising object storage cost across regions and cloud providers rather than across storage classes within a single provider, which reported up to 6x cost savings over commercial multi-region baselines. Despite being a peer reviewed, production-validated optimisation system that directly tests the premise of the research question “workload-aware optimisation > static configuration”, both TierBase and SkyStore are limited to the optimisation decision – rather than predicting future changes in the cost curve, nor estimating the actual size of the resulting saving. This research brings the missing piece of top-tier systems papers – that forward-looking forecasting component. TierBase and SkyStore, therefore, set it straight that it is realistic and useful to optimise with cost awareness and workload considerations in production and this is so only in hindsight or in retrospect and never in prediction. The validity of the forecasting layer that this proposal proposes is a different question and will be discussed in the cloud FinOps and forecasting literature next.

2.4 Predictive Cost Forecasting for Cloud FinOps

In Applied Intelligence (Springer, JCR-indexed), Beck, Dovern and Vogl (2025) examine how time-series forecasting is used in macroeconomic and financial applications and demonstrate that a substantial proportion of all forecasting research centers only on complex models, and not on any of these models against a naive forecast, while only a handful of models perform better than a naive forecast on series with low predictability. They have a different set of test domains, macroeconomics and finance, but the methodological care taken is exactly as relevant to the research question addressed here as it was in their domain of interest, and there is a forecasting component to any test of value only if it is demonstrated to be superior to a naive persistence baseline, not just to seem correct in isolation. This directly feeds into the evaluation design of the proposed framework (Section 3.3) which first tests the ability of the forecasting model against a naive persistence baseline to give a credit for actual forecasting ability, and not just headline accuracy. Bodra (2025) and Yakkanti (2025) separately make the case for cloud financial governance to be dominated by the use of resource-allocation and cost-optimisation algorithms based on machine learning, not as a methodology for forecasting, but rather as a practice. Both papers are contrasted, with Bodra’s work being a comprehensive performance study of a myriad of ML-based allocation algorithms over cloud resources more generally, and Yakkanti’s being closer to the FinOps process but a bit more limited in scope; neither of these papers suggests or evaluates a working pipeline that predicts allocation for storage-Class specifically, and thus their claims are aspirational and not demonstrated in this context. While the forecasting literature provides a methodological safety net (Beck, Dovern and Vogl, 2025), and FinOps provides a practice-level motivation (Bodra, 2025; Yakkanti, 2025), neither is applied, nor is their combination used to place and optimise the cost of Amazon S3 storage classes. These four themes are integrated into the specific research niche in which this proposal falls into, in Section 2.5.

2.5 Research Niche and Expected Contribution

In all the four themes above three recurring gaps are identified that relate directly to the research question formulated in Section 1. AWS’s tooling (Section 2.1) is reactive, and provides no forward estimate on cost savings, so it’s not designed to answer the question of whether or not a proactive alternative would save cost. Second, the ML systems literature (Sections 2.2–2.3) shows, across four different top-tier, peer-reviewed venues (MLSys, FAST, ICDE, VLDB), that learned, workload-aware placement and cost optimisation outperforms static rules and reactive tooling, but the evidence is validated at the level of

storage hierarchies, key-value stores, or cross-cloud placement, and never as an Amazon S3-native, per-object recommendation service that a practitioner could directly adopt. Third, the forecasting literature and FinOps literature (Section 2.4) confirms the importance of the concept of predictive cost management in principle, and provides some interesting methodological challenges because the literature is full of claims of high predictive accuracy that one should be skeptical of—hiding the details of the prediction process away from other teams, and not testing forecasting, storage-class recommendation and savings estimation together in one pipeline, not to mention specifically testing the same concept on AWS S3. This proposal’s niche is to fill the intersection of these three gaps: an AWS-native framework that recommends a storage class, forecasting what that storage class will cost in the future, and estimating savings of switching – but as Beck, Dovern and Vogl caution (2025), benchmarked against both AWS-native tooling and a naive persistence baseline. The above works, each illustrate a single of these pieces of the puzzle: learned placement, tiering, or cross-cloud cost optimisation, and the works of Bodra (2025) and Yakkanti (2025) call for predictive FinOps, but without providing an empirically validated Amazon S3 pipeline for recommendation, forecasting and savings estimation. To answer the research question, therefore, requires original empirical work, a work that Section 3 aims to provide.

3 Research Method and Specification

3.1 Proposed Solution and System Architecture

The five components of the framework are connected and brought into existence with the deployment strategy architecture built, with the addition of an explicit forecasting and savings-estimation layer. The following Table 1 summarises each component and Figure 2 shows the end-to-end data flow from the S3 bucket under test to the evaluation and reporting layer.

Table 1: Framework Components

Component	Function	Key Technology
1. Metadata Collection Module	Extracts per-object attributes: size, age, storage class, access frequency	AWS SDK (Boto3), S3 Inventory, CloudWatch metrics
2. Recommendation Engine	Recommends the most cost-appropriate storage class per object	Rule layer (AWS pricing thresholds) + supervised ML classifier (Random Forest / XGBoost)
3. Cost Forecasting Module	Projects future storage cost under current vs. recommended configuration	Time-series model (Prophet/ARIMA), benchmarked against a naive persistence baseline
4. Savings Estimation Component	Quantifies expected saving (USD, %) before a recommendation is applied	Cost-delta calculation using AWS Pricing API
5. Evaluation & Reporting Layer	Benchmarks framework decisions against native AWS baselines	Python (pandas), matplotlib/seaborn, statistical testing

3.2 Tools, Technologies and Test Data

- The programming language is Python 3.11, and the ML stack includes the recommender engine (XGBoost) and forecasting (Prophet and statsmodels) engines, along with the Boto3 API.

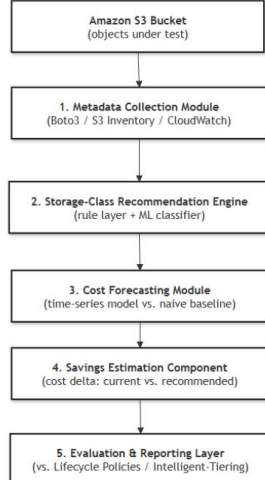


Figure 1: E2E: End to End Framework Architecture

- Visualisation & Reproducibility: pandas, matplotlib/seaborn, Jupyter notebooks and Git version control.
- Test data: a controlled S3 bucket containing objects of different size and age, with varying access patterns, but also enriched with historical AWS pricing data for the S3 service to assess the accuracy of the forecasts in comparison to real S3 billing data.

3.3 Evaluation Plan

The answer to the research question is presented by comparing the recommendations, forecasts and realised savings of the framework against AWS-native baselines for three representative. The described workloads (static/archival, mixed-access, high-churn), shown in Table 2.

Table 2: Evaluation Metrics

Metric	Definition	Compared Against
Storage-class allocation accuracy	% of objects assigned the cost-optimal class, validated against a labelled test set	Current (unoptimised) configuration
Forecast error (MAPE / RMSE)	Deviation between forecast and actual next-period billed cost	Naive persistence baseline (per Beck et al., 2025)
Realised cost savings (USD, %)	Actual billed cost reduction after recommendations are applied	AWS Lifecycle Policies; AWS Intelligent-Tiering
Operational overhead	Compute time/cost of running the framework itself	Manual/administrator review time (qualitative)

A paired Wilcoxon signed-rank test will be used to determine statistical significance of the cost outcomes of the framework with each AWS-native baseline across repeated trial buckets ($\alpha = 0.05$). The Wilcoxon test is used when the number of trial buckets per workload type is expected to be small ($n \approx 10$ – 15 per workload), and since the cost differences between the two storage types are likely not normally distributed (AWS storage-cost differences are typically right skewed), the Wilcoxon test is an appropriate choice. Success is a statistically significant decrease in cost compared to both Lifecycle Policies and Intelligent-Tiering across at least two of the three workload types, and a forecast error that is statistically significantly better than the naive persistence baseline that was identified as a risk by Beck, Doovern, and Vogl (2025).

One limitation of forecast validation is that the data in AWS Cost Explorer is not real-time: AWS charges for usage are based on the previous day’s total usage, which can be up to 24 hours earlier, and AWS finalised billing may not be available for a further delay as usage is added to and settled. Same-day forecast accuracy will be validated against CloudWatch storage-utilisation and request-count metrics, as a near-real-time proxy for cost, with AWS Cost Explorer figures used for final, settled-cost confirmation only on the day each billing period closes, to avoid validating the forecasting module against stale or incomplete cost data.

3.4 Project Plan and Gantt Chart



Figure 2: Project Plan and Gantt Chart

3.5 Ethical Considerations

There are no personal or sensitive data involved in this research and no human participants. All test objects held in the experimental S3 bucket will be synthetic dummy files which is to say the content of the file is not analysed, only metadata (file size, storage class, file timestamps, file access counts). If the public pricing or usage information is utilized to validate the forecasting module, then only AWS public pricing history is used. The study is in the lowest-risk category in NCI’s ethics decision matrix (Public data: Yes; Private data: No; Human subjects: No) and accompanied by a signed Declaration of Ethics Consideration. AWS costs for experimentation will be constrained and monitored by budget alerts to prevent cost creep and documented separately on the Cloud Resources Requirements Form.

References

- [1] Beck, N., Dovern, J. and Vogl, S. (2025) 'Mind the Naive Forecast! A Rigorous Evaluation of Forecasting Models for Time Series with Low Predictability', *Applied Intelligence*, 55, article 395. <https://doi.org/10.1007/s10489-025-06268-w>
- [2] Bodra, D. and Khairnar, S. (2025) 'Machine Learning-Based Cloud Resource Allocation Algorithms: A Comprehensive Comparative Review', *Frontiers in Computer Science*, 7:1678976. <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1678976/full>
- [3] Liu, S., Mo, X., Hershcovitch, M., Zhang, H., Cheng, A., Girmonsky, G., Vernik, G., Factor, M., Bang, T., Ponnappalli, S., Crooks, N., Gonzalez, J.E., Harnik, D. and Stoica, I. (2025) 'SkyStore:

- Cost-Optimized Object Storage Across Regions and Clouds’, *Proceedings of the VLDB Endowment*, 18(7), pp. 2084–2096. <https://doi.org/10.14778/3734839.3734846>
- [4] Shen, Z., Yang, S., Chen, W., Wang, K., Li, Y., Jin, J., Jia, W., Chen, J., Su, Y., Duan, X., Chen, W., Wang, L., Song, J., Ruan, R. and Lin, X. (2025) ‘TierBase: A Workload-Driven Cost-Optimized Key-Value Store’, *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pp. 571–584. <https://arxiv.org/abs/2505.06556>
- [5] Tu, K., Wu, K., Arpaci-Dusseau, A.C. and Arpaci-Dusseau, R.H. (2026) ‘Getting the MOST out of your Storage Hierarchy with Mirror-Optimized Storage Tiering’, *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST ’26)*, pp. 561–578. <https://arxiv.org/abs/2512.03279>
- [6] Yakkanti, P.R. (2025) ‘AI-Enabled FinOps for Cloud Cost Optimization: Enhancing Financial Governance in Cloud Environments’, *European Journal of Computer Science and Information Technology*, 13(11), pp. 17–29. <https://eajournals.org/ejcsit/vol13-issue11-2025/ai-enabled-finops-for-cloud-cost-\protect\penalty\z@optimization-enhancing-financial-governance-in-cloud-environments/>
- [7] Yang, C., Li, Y., Maas, M., Uysal, M., Hafeez, U.U., Merchant, A. and McDougall, R. (2025) ‘A Bring-Your-Own-Model Approach for ML-Driven Storage Placement in Warehouse-Scale Computers’, *Proceedings of Machine Learning and Systems (MLSys 2025)*, 7. <https://arxiv.org/abs/2501.05651>